

Protein-Ligand Binding Residue Prediction using CNN

Yifeng Cui, Qiwen Dong, Daocheng Hong and Xikun Wang

Research Paper

Seminar

Abdus Salam Khazi

Albert-Ludwigs-Universität Freiburg

February 7, 2021



What are proteins?



Biological Background

What are proteins?

- They are complex molecules that are work-horses of a living organism. They are literally machines. [TEDx Talk](#)



Biological Background

What are proteins?

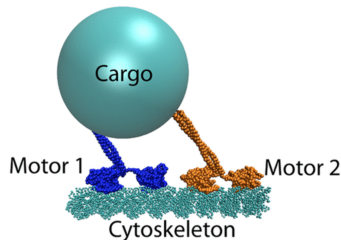
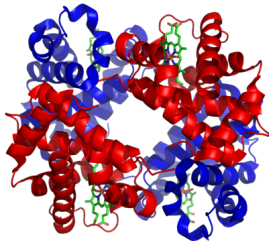
- They are complex molecules that are work-horses of a living organism. They are literally machines. [TEDx Talk](#)
- Proteins are essential for almost every function of a living organism's body. E.g: Movement of muscles, Transportation of oxygen, Digestion, Immune system etc...



Biological Background

What are proteins?

- They are complex molecules that are work-horses of a living organism. They are literally machines. [TEDx Talk](#)
- Proteins are essential for almost every function of a living organism's body. E.g: Movement of muscles, Transportation of oxygen, Digestion, Immune system etc...
- Below are 2 proteins - Hemoglobin(Left) and Motor Proteins(Right). Notice their shapes are different.



Biological Background

Protein formation process. DNA to Protein



Biological Background

Protein formation process. DNA to Protein

- All proteins required by an organism are encoded in the DNA.



Protein formation process. DNA to Protein

- All proteins required by an organism are encoded in the DNA.
- First, RNA polymerase transcribes the part of our DNA which contains protein encoding to messenger RNA a.k.a mRNA.



Protein formation process. DNA to Protein

- All proteins required by an organism are encoded in the DNA.
- First, RNA polymerase transcribes the part of our DNA which contains protein encoding to messenger RNA a.k.a mRNA.
- mRNA is post-processed to get an RNA. This RNA goes outside the nucleus of the cell. $\text{RNA} \in \{A, T, C, G\}^n$ where $n \in \mathbb{I}^+$



Protein formation process. DNA to Protein

- All proteins required by an organism are encoded in the DNA.
- First, RNA polymerase transcribes the part of our DNA which contains protein encoding to messenger RNA a.k.a mRNA.
- mRNA is post-processed to get an RNA. This RNA goes outside the nucleus of the cell. $\text{RNA} \in \{A, T, C, G\}^n$ where $n \in \mathbb{I}^+$
- Ribosomes in the cell cytoplasm then translate RNA to an amino acid sequence. This sequence is called primary structure of the protein. The mapping can be written as -
 $\text{Ribosome} : \{A, T, C, G\}^n \rightarrow \{R, N, D, C \dots\}^m$
where $m \approx \frac{n}{3}$ and $m \in \mathbb{I}^+$.



Protein formation process. DNA to Protein

- All proteins required by an organism are encoded in the DNA.
- First, RNA polymerase transcribes the part of our DNA which contains protein encoding to messenger RNA a.k.a mRNA.
- mRNA is post-processed to get an RNA. This RNA goes outside the nucleus of the cell. $\text{RNA} \in \{A, T, C, G\}^n$ where $n \in \mathbb{I}^+$
- Ribosomes in the cell cytoplasm then translate RNA to an amino acid sequence. This sequence is called primary structure of the protein. The mapping can be written as -
 $\text{Ribosome} : \{A, T, C, G\}^n \rightarrow \{R, N, D, C \dots\}^m$
where $m \approx \frac{n}{3}$ and $m \in \mathbb{I}^+$.
- This amino acid sequence folds into a 3D shape depending on its sequence. This structure is called the tertiary protein structure.



What are Ligands?



Biological Background

What are Ligands?

- They are molecules that bind to particular proteins, called receptor proteins. They signal the proteins to start a process.
- Example - Glucagon is a ligand created by pancreas. This binds to Glucagon receptor protein which signals it to control the blood sugar level.



Biological Background

What are Ligands?

- They are molecules that bind to particular proteins, called receptor proteins. They signal the proteins to start a process.
- Example - Glucagon is a ligand created by pancreas. This binds to Glucagon receptor protein which signals it to control the blood sugar level.

Why study protein Ligand Interaction?



What are Ligands?

- They are molecules that bind to particular proteins, called receptor proteins. They signal the proteins to start a process.
- Example - Glucagon is a ligand created by pancreas. This binds to Glucagon receptor protein which signals it to control the blood sugar level.

Why study protein Ligand Interaction?

- It will help us understand biological processes better.
- In the above example we can develop better treatment for type-2 diabetes by understanding the working of Glucagon and its receptor protein.



What is our problem?

- In this paper the authors are trying to estimate the function $f : \alpha^m \rightarrow \{Binding, NonBinding\}^m$ where m is the length of any ligand binding protein and $\alpha \in \{\text{Set of amino acids possible in a protein}\}$.



What is our problem?

- In this paper the authors are trying to estimate the function $f : \alpha^m \rightarrow \{Binding, NonBinding\}^m$ where m is the length of any ligand binding protein and $\alpha \in \{\text{Set of amino acids possible in a protein}\}$.
- They use Deep Convolutional Neural Networks for this purpose.



What is our problem?

- In this paper the authors are trying to estimate the function $f : \alpha^m \rightarrow \{Binding, NonBinding\}^m$ where m is the length of any ligand binding protein and $\alpha \in \{\text{Set of amino acids possible in a protein}\}$.
- They use Deep Convolutional Neural Networks for this purpose.
- This function is learnable because the binding sites are directly influenced by the tertiary protein structure which in turn depends on its primary structure.



What is our problem?

- In this paper the authors are trying to estimate the function $f : \alpha^m \rightarrow \{Binding, NonBinding\}^m$ where m is the length of any ligand binding protein and $\alpha \in \{\text{Set of amino acids possible in a protein}\}$.
- They use Deep Convolutional Neural Networks for this purpose.
- This function is learnable because the binding sites are directly influenced by the tertiary protein structure which in turn depends on its primary structure.
- Notice that m is variable here!!!



What is our problem?

- In this paper the authors are trying to estimate the function $f : \alpha^m \rightarrow \{Binding, NonBinding\}^m$ where m is the length of any ligand binding protein and $\alpha \in \{\text{Set of amino acids possible in a protein}\}$.
- They use Deep Convolutional Neural Networks for this purpose.
- This function is learnable because the binding sites are directly influenced by the tertiary protein structure which in turn depends on its primary structure.
- Notice that m is variable here!!!
- Previous attempts to solve this problem have used SVM, Deep Neural networks, Bayes classifier, 3D Template matching (albeit with high computational costs).



What is our problem? - Continued

- Previous methods used sliding window concept to capture residue dependencies.



What is our problem? - Continued

- Previous methods used sliding window concept to capture residue dependencies.
- But, nearby residues in the tertiary structure may be located far away in its primary structures.
- Just increasing window size is infeasible due to memory and computation costs.



What is our problem? - Continued

- Previous methods used sliding window concept to capture residue dependencies.
- But, nearby residues in the tertiary structure may be located far away in its primary structures.
- Just increasing window size is infeasible due to memory and computation costs.
- The authors create a CNN architecture (named DeepCSeqSite) and use it as a universal function approximator.



What is our problem? - Continued

- Previous methods used sliding window concept to capture residue dependencies.
- But, nearby residues in the tertiary structure may be located far away in its primary structures.
- Just increasing window size is infeasible due to memory and computation costs.
- The authors create a CNN architecture (named DeepCSeqSite) and use it as a universal function approximator.
- The authors do not solve ligand specific prediction tasks.



What is our problem? - Continued

- Previous methods used sliding window concept to capture residue dependencies.
- But, nearby residues in the tertiary structure may be located far away in its primary structures.
- Just increasing window size is infeasible due to memory and computation costs.
- The authors create a CNN architecture (named DeepCSeqSite) and use it as a universal function approximator.
- The authors do not solve ligand specific prediction tasks.
- They classify each amino-acid in the sequence as binding or non-binding.



Binding-Residue Prediction vs NMT



Binding-Residue Prediction vs NMT

- Neural Machine Translation (NMT) can be defined as a function $g : \alpha^n \rightarrow \beta^m$ where α, β are alphabets of 2 distinct languages and $n, m \in \mathbb{I}^+$



Binding-Residue Prediction vs NMT

- Neural Machine Translation (NMT) can be defined as a function $g : \alpha^n \rightarrow \beta^m$ where α, β are alphabets of 2 distinct languages and $n, m \in \mathbb{I}^+$
- Binding-Site prediction is a specific case of NMT, where $m = n$, α is one of 20 amino acids and β is $\{Binding, NonBinding\}$.



Binding-Residue Prediction vs NMT

- Neural Machine Translation (NMT) can be defined as a function $g : \alpha^n \rightarrow \beta^m$ where α, β are alphabets of 2 distinct languages and $n, m \in \mathbb{I}^+$
- Binding-Site prediction is a specific case of NMT, where $m = n$, α is one of 20 amino acids and β is $\{Binding, NonBinding\}$.
- The local and long distance coorelation between different α^n and β^m exist in both NMT and BR prediction.



Binding-Residue Prediction vs NMT

- Neural Machine Translation (NMT) can be defined as a function $g : \alpha^n \rightarrow \beta^m$ where α, β are alphabets of 2 distinct languages and $n, m \in \mathbb{I}^+$
- Binding-Site prediction is a specific case of NMT, where $m = n$, α is one of 20 amino acids and β is $\{Binding, NonBinding\}$.
- The local and long distance coorelation between different α^n and β^m exist in both NMT and BR prediction.
- Hence, any solution of NMT can be used for our problem.



Binding-Residue Prediction vs NMT

- Neural Machine Translation (NMT) can be defined as a function $g : \alpha^n \rightarrow \beta^m$ where α, β are alphabets of 2 distinct languages and $n, m \in \mathbb{I}^+$
- Binding-Site prediction is a specific case of NMT, where $m = n$, α is one of 20 amino acids and β is $\{Binding, NonBinding\}$.
- The local and long distance correlation between different α^n and β^m exist in both NMT and BR prediction.
- Hence, any solution of NMT can be used for our problem.
- Fun fact - Ribosome is also "Translating" from DNA alphabet to Amino acid alphabet !!!



RNNs as a solution



RNNs as a solution

- As our amino acid sequence is variable in length, we could use RNNs to capture long term dependencies in the sequence.



RNNs as a solution

- As our amino acid sequence is variable in length, we could use RNNs to capture long term dependencies in the sequence.
- There are some disadvantages



RNNs as a solution

- As our amino acid sequence is variable in length, we could use RNNs to capture long term dependencies in the sequence.
- There are some disadvantages
 - The first amino acid is processed n times, where n is the length of the protein. And the last amino acid is processed only 1 time.



RNNs as a solution

- As our amino acid sequence is variable in length, we could use RNNs to capture long term dependencies in the sequence.
- There are some disadvantages
 - The first amino acid is processed n times, where n is the length of the protein. And the last amino acid is processed only 1 time.
 - RNNs are sequential in principle. Parallelizing the computation is not possible.



RNNs as a solution

- As our amino acid sequence is variable in length, we could use RNNs to capture long term dependencies in the sequence.
- There are some disadvantages
 - The first amino acid is processed n times, where n is the length of the protein. And the last amino acid is processed only 1 time.
 - RNNs are sequential in principle. Parallelizing the computation is not possible.

Solution - Convolution Neural Networks.



RNNs as a solution

- As our amino acid sequence is variable in length, we could use RNNs to capture long term dependencies in the sequence.
- There are some disadvantages
 - The first amino acid is processed n times, where n is the length of the protein. And the last amino acid is processed only 1 time.
 - RNNs are sequential in principle. Parallelizing the computation is not possible.

Solution - Convolution Neural Networks.

- Temporal-CNNs can be parallelized, are 10x faster and give state of the art results.



RNNs as a solution

- As our amino acid sequence is variable in length, we could use RNNs to capture long term dependencies in the sequence.
- There are some disadvantages
 - The first amino acid is processed n times, where n is the length of the protein. And the last amino acid is processed only 1 time.
 - RNNs are sequential in principle. Parallelizing the computation is not possible.

Solution - Convolution Neural Networks.

- Temporal-CNNs can be parallelized, are 10x faster and give state of the art results.
- In Temporal-CNN (DeepCSeqSite) architecture, both local and global dependencies can be captured by increasing the number of CNN layers. More on this in later slides!!



CNNs as a solution

- Let $\alpha \in \{\text{Ala, Arg, Asn ...}\}$. i.e Set of amino acids.



CNNs as a solution

- Let $\alpha \in \{\text{Ala, Arg, Asn ...}\}$. i.e Set of amino acids.
- A protein of length m is represented as α^m



CNNs as a solution

- Let $\alpha \in \{\text{Ala, Arg, Asn ...}\}$. i.e Set of amino acids.
- A protein of length m is represented as α^m
- A classifier that classifies α^m to $\{\text{Binding, NonBinding}\}^m$ is trained.



CNNs as a solution

- Let $\alpha \in \{\text{Ala, Arg, Asn ...}\}$. i.e Set of amino acids.
- A protein of length m is represented as α^m
- A classifier that classifies α^m to $\{\text{Binding, NonBinding}\}^m$ is trained.
- During training, a mini-batch of B proteins is taken together. Each protein α^m in the batch is padded with p_i dummy residues given by

$$p_i = \max(\{m_1, m_2 \cdots m_B\}) - m_i$$

to make the lengths of proteins in a single batch equal.



CNNs as a solution

- Let $\alpha \in \{\text{Ala}, \text{Arg}, \text{Asn} \dots\}$. i.e Set of amino acids.
- A protein of length m is represented as α^m
- A classifier that classifies α^m to $\{\text{Binding}, \text{NonBinding}\}^m$ is trained.
- During training, a mini-batch of B proteins is taken together. Each protein α^m in the batch is padded with p_i dummy residues given by

$$p_i = \max(\{m_1, m_2 \dots m_B\}) - m_i$$

to make the lengths of proteins in a single batch equal.

- The padding is removed before loss calculation.



CNNs as a solution

- Let $\alpha \in \{\text{Ala, Arg, Asn ...}\}$. i.e Set of amino acids.
- A protein of length m is represented as α^m
- A classifier that classifies α^m to $\{\text{Binding, NonBinding}\}^m$ is trained.
- During training, a mini-batch of B proteins is taken together. Each protein α^m in the batch is padded with p_i dummy residues given by

$$p_i = \max(\{m_1, m_2 \cdots m_B\}) - m_i$$

to make the lengths of proteins in a single batch equal.

- The padding is removed before loss calculation.
- The lengths of the batches can differ from each other.



CNNs as a solution

- Each amino-acid in a protein is mapped to $\mathbb{R}^{30}$. Thus a protein is mapped to a feature space $\mathbb{R}^{m \times 30}$



CNNs as a solution

- Each amino-acid in a protein is mapped to $\mathbb{R}^{30}$. Thus a protein is mapped to a feature space $\mathbb{R}^{m \times 30}$
- $\mathbb{R}^{30}$ are manually engineered features given by
 - **Position-Specific Score Matrix (PSSM)** $\in \mathbb{R}^{20}$. This is the probability of mutation to other amino acids at each position. Since this mutation is not position independent, it is calculated for the whole protein i.e $\in \mathbb{R}^{m \times 20}$. Normalized values are calculated by

$$Norm(\mathbf{x}) = \frac{1}{1 + 2^{-\mathbf{x}}}; \mathbf{x}, \in \mathbb{R}^{m \times 20}$$



CNNs as a solution

- Each amino-acid in a protein is mapped to $\mathbb{R}^{30}$. Thus a protein is mapped to a feature space $\mathbb{R}^{m \times 30}$
- $\mathbb{R}^{30}$ are manually engineered features given by
 - **Position-Specific Score Matrix (PSSM)** $\in \mathbb{R}^{20}$. This is the probability of mutation to other amino acids at each position. Since this mutation is not position independent, it is calculated for the whole protein i.e $\in \mathbb{R}^{m \times 20}$. Normalized values are calculated by

$$Norm(\mathbf{x}) = \frac{1}{1 + 2^{-\mathbf{x}}}; \mathbf{x}, \in \mathbb{R}^{m \times 20}$$

- **Relative solvent accessibility** $\in \mathbb{R}$. Buried (< 0.25) or exposed (> 0.25) in the tertiary-structure.



- Each amino-acid in a protein is mapped to $\mathbb{R}^{30}$. Thus a protein is mapped to a feature space $\mathbb{R}^{m \times 30}$
- $\mathbb{R}^{30}$ are manually engineered features given by
 - **Position-Specific Score Matrix (PSSM)** $\in \mathbb{R}^{20}$. This is the probability of mutation to other amino acids at each position. Since this mutation is not position independent, it is calculated for the whole protein i.e $\in \mathbb{R}^{m \times 20}$. Normalized values are calculated by

$$Norm(\mathbf{x}) = \frac{1}{1 + 2^{-\mathbf{x}}}; \mathbf{x}, \in \mathbb{R}^{m \times 20}$$

- **Relative solvent accessibility** $\in \mathbb{R}$. Buried (< 0.25) or exposed (> 0.25) in the tertiary-structure.
- **Secondary Structure** i.e $[\alpha\text{-helix } \beta\text{-strand coil}]^T \in \mathbb{R}^3$. $\alpha\text{-helix}, \beta\text{-strand}, \text{coil} \in [0, 1]$ indicating probabilities of existence.



- Each amino-acid in a protein is mapped to $\mathbb{R}^{30}$. Thus a protein is mapped to a feature space $\mathbb{R}^{m \times 30}$
- $\mathbb{R}^{30}$ are manually engineered features given by
 - **Position-Specific Score Matrix (PSSM)** $\in \mathbb{R}^{20}$. This is the probability of mutation to other amino acids at each position. Since this mutation is not position independent, it is calculated for the whole protein i.e $\in \mathbb{R}^{m \times 20}$. Normalized values are calculated by

$$Norm(\mathbf{x}) = \frac{1}{1 + 2^{-\mathbf{x}}}; \mathbf{x}, \in \mathbb{R}^{m \times 20}$$

- **Relative solvent accessibility** $\in \mathbb{R}$. Buried (< 0.25) or exposed (> 0.25) in the tertiary-structure.
- **Secondary Structure** i.e $[\alpha\text{-helix } \beta\text{-strand coil}]^T \in \mathbb{R}^3$. $\alpha\text{-helix}, \beta\text{-strand}, \text{coil} \in [0, 1]$ indicating probabilities of existence.
- **Dihedral angle** - $\mathbf{A} \in \mathbb{R}^2$ where $Norm(\mathbf{A}) = \frac{\mathbf{A}}{360}$.



- **Conservation rating score** during evolution.
[Relative-entropy Jensen-Shannon-divergence]^T $\in \mathbb{R}^2$.



- **Conservation rating score** during evolution.
[Relative-entropy Jensen-Shannon-divergence]^T $\in \mathbb{R}^2$.
- **Residue type** $\in \mathbb{R}$ given by

$$r = \frac{type}{21}; type \in [0, 20]; type \in \mathbb{I}^+$$

Here 0 means dummy residue, [1 – 20] represents other residues.



- **Conservation rating score** during evolution.
[Relative-entropy Jensen-Shannon-divergence] $^T \in \mathbb{R}^2$.
- **Residue type** $\in \mathbb{R}$ given by

$$r = \frac{type}{21}; type \in [0, 20]; type \in \mathbb{I}^+$$

Here 0 means dummy residue, $[1 - 20]$ represents other residues.

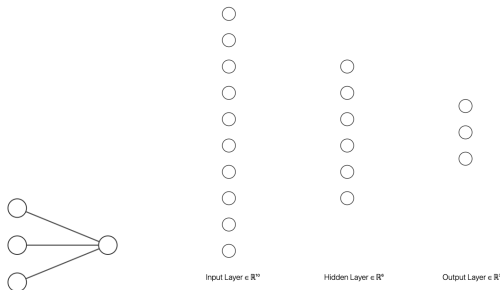
- **Position embeddings** $\in \mathbb{R}$ given by

$$PE_i = \frac{i}{L}$$

for residue at i^{th} position where L is the length of the protein.



Effective Context Scope / Input Field



Consider the above kernel with width 3 (left) and a neural network (right). Let **I**, **H** and **O** denote the input, hidden and output vectors/layers respectively. Let $\mathbf{I}_i$ denote a single node in the input vector. Here $\mathbf{I}_0$ is the top element in the input vector and in the image.



Effective Context Scope / Input Field

When we apply Kernel to both **I** and **H** with stride 1,



Effective Context Scope / Input Field

When we apply Kernel to both **I** and **H** with stride 1,

$$\text{Context}(\mathbf{O}_0) = \{H_0, H_1, H_2\}$$

$$\text{InputField}(\mathbf{H}_0) = \{I_0, I_1, I_2\}$$

$$\text{InputField}(\mathbf{H}_1) = \{I_1, I_2, I_3\}$$

$$\text{InputField}(\mathbf{H}_2) = \{I_2, I_3, I_4\}$$

$$\text{InputField}(\mathbf{O}_0) = \text{InputField}(\mathbf{H}_0) \cup \text{InputField}(\mathbf{H}_1) \cup \text{InputField}(\mathbf{H}_2)$$

$$\text{InputField}(\mathbf{O}_0) = \{I_0, I_1, I_2, I_3, I_4\}$$

$$|\text{InputField}(\mathbf{O}_0)| = 5 = 1 + 2(3 - 1) = 1 + n(k - 1)$$



Effective Context Scope / Input Field

When we apply Kernel to both **I** and **H** with stride 1,

$$\text{Context}(\mathbf{O}_0) = \{H_0, H_1, H_2\}$$

$$\text{InputField}(\mathbf{H}_0) = \{I_0, I_1, I_2\}$$

$$\text{InputField}(\mathbf{H}_1) = \{I_1, I_2, I_3\}$$

$$\text{InputField}(\mathbf{H}_2) = \{I_2, I_3, I_4\}$$

$$\text{InputField}(\mathbf{O}_0) = \text{InputField}(\mathbf{H}_0) \cup \text{InputField}(\mathbf{H}_1) \cup \text{InputField}(\mathbf{H}_2)$$

$$\text{InputField}(\mathbf{O}_0) = \{I_0, I_1, I_2, I_3, I_4\}$$

$$|\text{InputField}(\mathbf{O}_0)| = 5 = 1 + 2(3 - 1) = 1 + n(k - 1)$$

- Hence a single convolutional node in n^{th} convolutional layer (depth) with kernel width k and stride 1 has an input field $1 + n(k - 1)$.



Effective Context Scope / Input Field

When we apply Kernel to both **I** and **H** with stride 1,

$$\text{Context}(\mathbf{O}_0) = \{H_0, H_1, H_2\}$$

$$\text{InputField}(\mathbf{H}_0) = \{I_0, I_1, I_2\}$$

$$\text{InputField}(\mathbf{H}_1) = \{I_1, I_2, I_3\}$$

$$\text{InputField}(\mathbf{H}_2) = \{I_2, I_3, I_4\}$$

$$\text{InputField}(\mathbf{O}_0) = \text{InputField}(\mathbf{H}_0) \cup \text{InputField}(\mathbf{H}_1) \cup \text{InputField}(\mathbf{H}_2)$$

$$\text{InputField}(\mathbf{O}_0) = \{I_0, I_1, I_2, I_3, I_4\}$$

$$|\text{InputField}(\mathbf{O}_0)| = 5 = 1 + 2(3 - 1) = 1 + n(k - 1)$$

- Hence a single convolutional node in n^{th} convolutional layer (depth) with kernel width k and stride 1 has an input field $1 + n(k - 1)$.
- The author propose an increase in n i.e the depth of CNN instead of k to prevent huge growth in optimization parameters.



Basic Design - Standard DeepCSeq-Site

Here we consider batch size $B = 1$ for simplicity. Preprocessed protein $\mathbf{P} \in \mathbb{R}^{m \times d}$ is input to the DeepCSeq network. Here $d = 30$ and m is the length of the protein in the batch.



Basic Design - Standard DeepCSeq-Site

Here we consider batch size $B = 1$ for simplicity. Preprocessed protein $\mathbf{P} \in \mathbb{R}^{m \times d}$ is input to the DeepCSeq network. Here $d = 30$ and m is the length of the protein in the batch.

- Let Kernel $\mathbf{K} \in \mathbb{R}^{k \times d}$ be the 1-D kernel in our convolutions.



Basic Design - Standard DeepCSeq-Site

Here we consider batch size $B = 1$ for simplicity. Preprocessed protein $\mathbf{P} \in \mathbb{R}^{m \times d}$ is input to the DeepCSeq network. Here $d = 30$ and m is the length of the protein in the batch.

- Let Kernel $\mathbf{K} \in \mathbb{R}^{k \times d}$ be the 1-D kernel in our convolutions.
- $\mathbf{0}^{\frac{k}{2} \times d}$ is padded at both ends of $\mathbf{P}$ to maintain length m . This padding is also done at each step throughout the network for the same reason.



Here we consider batch size $B = 1$ for simplicity. Preprocessed protein $\mathbf{P} \in \mathbb{R}^{m \times d}$ is input to the DeepCSeq network. Here $d = 30$ and m is the length of the protein in the batch.

- Let Kernel $\mathbf{K} \in \mathbb{R}^{k \times d}$ be the 1-D kernel in our convolutions.
- $\mathbf{0}_{\frac{k}{2} \times d}$ is padded at both ends of $\mathbf{P}$ to maintain length m . This padding is also done at each step throughout the network for the same reason.
- To capture local co-relation and covalent bonds between adjacent residues, the following convolution is applied

$$[\mathbf{K}_1(\mathbf{P}) \ \mathbf{K}_2(\mathbf{P}) \ \cdots \ \mathbf{K}_{2c}(\mathbf{P})]^T \in \mathbb{R}^{m \times 1 \times 2c}$$



Here we consider batch size $B = 1$ for simplicity. Preprocessed protein $\mathbf{P} \in \mathbb{R}^{m \times d}$ is input to the DeepCSeq network. Here $d = 30$ and m is the length of the protein in the batch.

- Let Kernel $\mathbf{K} \in \mathbb{R}^{k \times d}$ be the 1-D kernel in our convolutions.
- $\mathbf{0}_{\frac{k}{2} \times d}$ is padded at both ends of $\mathbf{P}$ to maintain length m . This padding is also done at each step throughout the network for the same reason.
- To capture local co-relation and covalent bonds between adjacent residues, the following convolution is applied

$$[\mathbf{K}_1(\mathbf{P}) \ \mathbf{K}_2(\mathbf{P}) \ \cdots \ \mathbf{K}_{2c}(\mathbf{P})]^T \in \mathbb{R}^{m \times 1 \times 2c}$$

- $\mathbb{R}^{m \times 1 \times 2c}$ is input to N Basic blocks which form the encoder of the network.



Basic Block - DeepCSeq-Site

A basic block in the network is defined as

$$\text{Block} : \mathbb{R}^{m \times 1 \times 2c} \rightarrow \mathbb{R}^{m \times 1 \times 2c}$$



Basic Block - DeepCSeq-Site

A basic block in the network is defined as

$$\text{Block} : \mathbb{R}^{m \times 1 \times 2c} \rightarrow \mathbb{R}^{m \times 1 \times 2c}$$

A series of N blocks are applied in the following form

$$\text{Block}_N(\text{Block}_{N-1} \dots \text{Block}_2(\text{Block}_1(\mathbf{X}))); \mathbf{X} \in \mathbb{R}^{m \times 1 \times 2c}$$



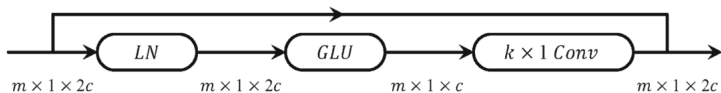
A basic block in the network is defined as

$$\text{Block} : \mathbb{R}^{m \times 1 \times 2c} \rightarrow \mathbb{R}^{m \times 1 \times 2c}$$

A series of N blocks are applied in the following form

$$\text{Block}_N(\text{Block}_{N-1} \dots \text{Block}_2(\text{Block}_1(\mathbf{X}))); \mathbf{X} \in \mathbb{R}^{m \times 1 \times 2c}$$

The following diagram shows the basic architecture of a basic block



Basic Block - DeepCSeq-Site

- First input $\mathbf{X} \in \mathbb{R}^{m \times 1 \times 2c}$ is layer normalized to get $\mathbf{X}_{norm}$

$$\mathbf{x}_i = \frac{\mathbf{x}_i - \text{mean}(\mathbf{x}_i)}{\text{StdDeviation}(\mathbf{x}_i)}; \forall \mathbf{x}_i \in \mathbb{R}^{m \times 1}; i \in [1, 2c], \mathbb{I}^+$$



- First input $\mathbf{X} \in \mathbb{R}^{m \times 1 \times 2c}$ is layer normalized to get $\mathbf{X}_{norm}$

$$\mathbf{X}_i = \frac{\mathbf{X}_i - \text{mean}(\mathbf{X}_i)}{\text{StdDeviation}(\mathbf{X}_i)}; \forall \mathbf{X}_i \in \mathbb{R}^{m \times 1}; i \in [1, 2c], \mathbb{I}^+$$

- Next, $\mathbf{X}_{norm} = [\mathbf{A} : \mathbf{B}]$ where $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times 1 \times c}$ is passed through a Gated Linear Unit which performs the following operation to get $\mathbf{X}_{gated}$

$$GLU : \mathbb{R}^{m \times 1 \times 2c} \rightarrow \mathbb{R}^{m \times 1 \times c}$$

$$\mathbf{X}_{gated} = GLU([\mathbf{A} : \mathbf{B}]) = \mathbf{A} \odot \text{sigmoid}(\mathbf{B})$$



- First input $\mathbf{X} \in \mathbb{R}^{m \times 1 \times 2c}$ is layer normalized to get $\mathbf{X}_{norm}$

$$\mathbf{X}_i = \frac{\mathbf{X}_i - \text{mean}(\mathbf{X}_i)}{\text{StdDeviation}(\mathbf{X}_i)}; \forall \mathbf{X}_i \in \mathbb{R}^{m \times 1}; i \in [1, 2c], \mathbb{I}^+$$

- Next, $\mathbf{X}_{norm} = [\mathbf{A} : \mathbf{B}]$ where $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times 1 \times c}$ is passed through a Gated Linear Unit which performs the following operation to get $\mathbf{X}_{gated}$

$$GLU : \mathbb{R}^{m \times 1 \times 2c} \rightarrow \mathbb{R}^{m \times 1 \times c}$$

$$\mathbf{X}_{gated} = GLU([\mathbf{A} : \mathbf{B}]) = \mathbf{A} \odot \text{sigmoid}(\mathbf{B})$$

- $\mathbf{K} \in \mathbb{R}^{k \times 1 \times c}$ 1-D kernels are applied to $\mathbf{X}_{gated}$ to get $\mathbf{X}_{output} \in \mathbb{R}^{m \times 1 \times 2c}$
$$\mathbf{X}_{output} = [\mathbf{K}_1(\mathbf{X}_{gated}) \cdots \mathbf{K}_{2c}(\mathbf{X}_{gated})]^T$$



Solutions for gradient issues

- There is a skip connection from the input of the block to the output. This solves the vanishing gradient issue. The block function can be defined as

$$\text{Block}(\mathbf{X}) = \mathbf{X} + \text{Convolution}(\text{GLU}(\text{Normalization}(\mathbf{X})))$$



Solutions for gradient issues

- There is a skip connection from the input of the block to the output. This solves the vanishing gradient issue. The block function can be defined as

$$\text{Block}(\mathbf{X}) = \mathbf{X} + \text{Convolution}(\text{GLU}(\text{Normalization}(\mathbf{X})))$$

- Normalization prevents unexpected gradient due to the skip connection summation.



2 Fully connected (convolution) layers are defined for the decoder in the following way

- **Layer 1** - $\mathbf{K}^{1 \times 1 \times 2c}$ kernels are applied to get the first convolution output $\mathbf{X}_{hidden} \in \mathbb{R}^{m \times 1 \times c}$.

$$\mathbf{X}_{hidden} = [\mathbf{K}_1(\mathbf{X}) \ \mathbf{K}_2(\mathbf{X}) \cdots \mathbf{K}_c(\mathbf{X})]^T; \mathbf{X} \in \mathbb{R}^{m \times 1 \times 2c};$$

Here drop-out regularization is used.



2 Fully connected (convolution) layers are defined for the decoder in the following way

- **Layer 1** - $\mathbf{K}^{1 \times 1 \times 2c}$ kernels are applied to get the first convolution output $\mathbf{X}_{hidden} \in \mathbb{R}^{m \times 1 \times c}$.

$$\mathbf{X}_{hidden} = [\mathbf{K}_1(\mathbf{X}) \ \mathbf{K}_2(\mathbf{X}) \cdots \mathbf{K}_c(\mathbf{X})]^T; \mathbf{X} \in \mathbb{R}^{m \times 1 \times 2c};$$

Here drop-out regularization is used.

- **Layer 2** - $\mathbf{K}^{1 \times 1 \times c}$ kernels are applied to get the final prediction $\mathbf{Y}$.

$$\mathbf{Y} = [\mathbf{K}_1(\mathbf{X}_{hidden}) \ \mathbf{K}_2(\mathbf{X}_{hidden})]^T; \mathbf{Y} \in \mathbb{R}^{m \times 1 \times 2};$$

$\mathbf{Y}$ is passed through a soft-max layer to get the values in the range $[0, 1]$.



Loss function - Standard DeepCSeq-Site

The loss function used is the negative cross entropy (for 2-way softmax classifier) defined for a single amino-acid i in the sequence as

$$J_i(\theta) = -\mathbf{Y}_i \cdot \log(\hat{\mathbf{Y}}_i); \mathbf{Y}_i, \hat{\mathbf{Y}}_i \in \mathbb{R}^2$$

$$y_i \in \{0, 1\}; \hat{y}_i \in [0, 1]; \sum \mathbf{Y}_i = \sum \hat{\mathbf{Y}}_i = 1$$

The final loss function for a protein-ligand prediction is given by

$$J(\theta) = \sum_i^m J_i(\theta) + \gamma * L2Norm(\theta)$$

After training is complete the classification threshold of softmax is set to a value < 0.5 because of non binding skew in data.



Decoder - Enhanced DeepCSeq-Site

The probability of being a binding amino acid depends on whether its neighbours are binding or not. So to facilitate this, the following enhanced decoder is proposed by the authors



Decoder - Enhanced DeepCSeq-Site

The probability of being a binding amino acid depends on whether its neighbours are binding or not. So to facilitate this, the following enhanced decoder is proposed by the authors

- Additional "Context prediction" is given as input to the enhanced decoder.



Decoder - Enhanced DeepCSeq-Site

The probability of being a binding amino acid depends on whether its neighbours are binding or not. So to facilitate this, the following enhanced decoder is proposed by the authors

- Additional "Context prediction" is given as input to the enhanced decoder.
- Using Left Context prediction
 - Let $\mathbf{p}$ be the context/prediction of our protein.



Decoder - Enhanced DeepCSeq-Site

The probability of being a binding amino acid depends on whether its neighbours are binding or not. So to facilitate this, the following enhanced decoder is proposed by the authors

- Additional "Context prediction" is given as input to the enhanced decoder.
- Using Left Context prediction
 - Let $\mathbf{p}$ be the context/prediction of our protein.
 - $|\mathbf{p}| = m$ where m is the length of the protein.



Decoder - Enhanced DeepCSeq-Site

The probability of being a binding amino acid depends on whether its neighbours are binding or not. So to facilitate this, the following enhanced decoder is proposed by the authors

- Additional "Context prediction" is given as input to the enhanced decoder.
- Using Left Context prediction
 - Let $\mathbf{p}$ be the context/prediction of our protein.
 - $|\mathbf{p}| = m$ where m is the length of the protein.
 - The last amino-acid is not at the left of any other amino-acid. So it is not necessary for calculation.



The probability of being a binding amino acid depends on whether its neighbours are binding or not. So to facilitate this, the following enhanced decoder is proposed by the authors

- Additional "Context prediction" is given as input to the enhanced decoder.
- Using Left Context prediction
 - Let $\mathbf{p}$ be the context/prediction of our protein.
 - $|\mathbf{p}| = m$ where m is the length of the protein.
 - The last amino-acid is not at the left of any other amino-acid. So it is not necessary for calculation.
 - Hence $\mathbf{p}$ is transformed by pre-pending 0s to form $\mathbf{p}' = [0 \ 0 \cdots k \text{ zeros}] : [p_1 \ p_2 \ \cdots p_{m-1}]$



The probability of being a binding amino acid depends on whether its neighbours are binding or not. So to facilitate this, the following enhanced decoder is proposed by the authors

- Additional "Context prediction" is given as input to the enhanced decoder.
- Using Left Context prediction
 - Let $\mathbf{p}$ be the context/prediction of our protein.
 - $|\mathbf{p}| = m$ where m is the length of the protein.
 - The last amino-acid is not at the left of any other amino-acid. So it is not necessary for calculation.
 - Hence $\mathbf{p}$ is transformed by pre-pending 0s to form $\mathbf{p}' = [0 \ 0 \cdots k \text{ zeros}] : [p_1 \ p_2 \ \cdots p_{m-1}]$
 - Kernels of size $\mathbf{K} \in \mathbb{R}^{k \times 1}$ are applied to $\mathbf{p}'$ to get $\mathbf{P}_{left} \in \mathbb{R}^{m \times 1 \times 2c}$

$$\mathbf{P}_{left} = [\mathbf{K}_1(\mathbf{p}') \ \mathbf{K}_2(\mathbf{p}') \ \cdots \mathbf{K}_{2c}(\mathbf{p}')]^T$$



- Usage of Right Context prediction is just a mirror image of the Left.



Decoder - Enhanced DeepCSeq-Site

- Usage of Right Context prediction is just a mirror image of the Left.
- P_{left} , P_{right} and the encoder output are all added to form enhanced encoder's output.



Decoder - Enhanced DeepCSeq-Site

- Usage of Right Context prediction is just a mirror image of the Left.
- P_{left} , P_{right} and the encoder output are all added to form enhanced encoder's output.
- During training, we can directly use labels as the input to context prediction usage process.



Decoder - Enhanced DeepCSeq-Site

- Usage of Right Context prediction is just a mirror image of the Left.
- P_{left} , P_{right} and the encoder output are all added to form enhanced encoder's output.
- During training, we can directly use labels as the input to context prediction usage process.
- During testing/future-use, first the standard decoder is used for prediction. The output of the standard decoder is given as the input to the enhanced decoder (After pre-processing as discussed above).



Decoder - Enhanced DeepCSeq-Site

- Usage of Right Context prediction is just a mirror image of the Left.
- P_{left} , P_{right} and the encoder output are all added to form enhanced encoder's output.
- During training, we can directly use labels as the input to context prediction usage process.
- During testing/future-use, first the standard decoder is used for prediction. The output of the standard decoder is given as the input to the enhanced decoder (After pre-processing as discussed above).
- Then the decoder used is exactly the same as the standard decoder.



Evaluation

Let TP , TN , FP and FN be the number of True positive, True negative, False positive and False negative values of our prediction respectively. 3 metrics are used for evaluation.

- Mathew's correlation coefficient, MCC

$$MCC = \frac{TP * TN - FP * FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

- Precision

$$precision = \frac{TP}{TP + FP}$$

- Recall

$$recall = \frac{TP}{TP + FN}$$



There are 3 main hyperparameters to optimize - number of basic blocks N , kernel width k and the number of channels c .

- Optimization of k, n is performed independently.
- The authors fix $N = 10$ while optimising k and $k = 3$ while optimising N .
- As per empirical evidence, $N = 10$, $k = 5$ and $c = 256$ gives best results.
- Also $\gamma = 0.2$ and drop out ratio 0.5 is used during training.
- The optimization is done through stochastic gradient descent
- To prevent data skew, the mini-batch selection should be such that the proportion of binding to non-binding residues remains roughly the same to that of the whole training data.



The learning rate consists of 3 stages. Each stage has exponential decay.

- 10^{-5} is used for first stage. This is to deal with very steep gradients. (Optimization step < 19999)
- 10^{-4} is used for second stage. This is to avoid falling into local minima that predicts all residues as negative. There is a huge skew towards negative (non-binding) residues in our data set. (Optimization step $\in [20000 \rightarrow 99999]$)
- $2 * 10^{-4}$ for the final stage till the end of the optimisation. (Optimization step > 99999)



- The results were compared with 3D structure based models as they are the models with best results - TM-SI, COF and COA
- Standard DeepCSeq-Site is better than these by 0.05 in MCC score and 15% in precision
- There is bad Recall score because of non-binding bias in the data sets.
- Enhanced DCSI improves MCC by ≈ 0.02 . This is because of inclusion of more stronger context prediction input to the decoder
- Data augmentation was not found to be suitable for the model due to high computational costs and low effect.



Discussion and conclusion

- Traditional SVM and ANN do not utilize data effectively. They have no feature learning, low generalization and representational capacity.
- Deep CNN are better models. They self learn hierarchical features and learn more abstract features at greater depths. Due to availability of massive sequencing data, generalization of the deep networks is possible.
- They have advantage of capturing long distance relationships at higher depths without depending on a particular input field window.
- Non binding data skew ($\approx 96\%$) is better handled in DCS-SI as compared to SVM and other base-line models. For example - generalization and convergence of support vectors learned by SVM is difficult.



Future Work

- Usage of attention mechanisms to capture long distance coorelations between amino-acids.
- Inclusion of more 3D structure data in the feature map.
- Usage of Generative adverserial networks to tackle quantitative disparity between 3D structure data and sequence data.

